Chapter 08: Pointers

ENG3091

Khiem Nguyen

Contents

1 Memory and addresses **2**
- 1.1 Every variable lives at an address . . . 2
- 1.2 The address-of operator `&` . . . 2
- 1.3 The dereference operator `*` . . . 2
- 1.4 One symbol, three meanings . . . 3

2 Pointers **3**
- 2.1 A pointer holds an address . . . 3
- 2.2 Your first pointer . . . 4
- 2.3 Declaring pointers . . . 4
- 2.4 Always initialize your pointers . . . 5
- 2.5 Changing the pointer, or what it points at . . . 5
- 2.6 Pointers behave much like references… . . . 6
- 2.7 …but a pointer is not a reference . . . 6
- 2.8 The type of `&x` . . . 6
- 2.9 The size of a pointer . . . 7

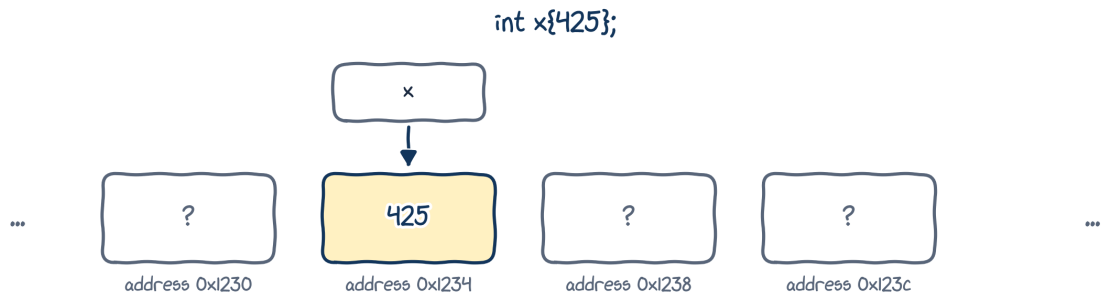
3 Null pointers **8**
- 3.1 `nullptr`: pointing at nothing . . . 8
- 3.2 Never dereference a null pointer . . . 8
- 3.3 Dangling pointers . . . 9

4 Pass by address **9**
- 4.1 Pass by address . . . 9
- 4.2 References or pointers? . . . 10

5 Wrapping up **10**
- 5.1 Summary . . . 10
- 5.2 Keep learning . . . 11

1 Memory and addresses

1.1 Every variable lives at an address



- When `int x{425};` runs, the program sets aside a piece of memory for `x`, at some **address**, say `0x1234`.
- Whenever we use `x`, the program goes to address `0x1234` and reads or writes the value stored there.
- The same is true for a reference: `int& ref{x};` is just another name for whatever lives at `0x1234`.

1.2 The address-of operator `&`

`&x` gives the **address** of `x`:

Listing 1 `address_of_operator_example.cpp`

```cpp
#include <iostream>

int main() {
    int x{5};
    std::cout << x << '\n';  // the value of x
    std::cout << &x << '\n'; // the address of x

    double d{42.0};
    std::cout << &d << '\n'; // the address of d
    return 0;
}
```

```
5
0x7ffd9bdba4ac
0x7ffd9bdba4b0
```

- Addresses are usually printed in hexadecimal. Yours will be different, and may change every run.
- An object bigger than one byte occupies several addresses; `&` gives the first one.

1.3 The dereference operator `*`

`*` does the opposite: given an address, it gives you the **object at that address**.

Listing 2 `dereference_operator_example.cpp`

```cpp
#include <iostream>

int main() {
    double x{5.0};
    std::cout << x << '\n';      // the value of x
    std::cout << &x << '\n';     // the address of x
    std::cout << *(&x) << '\n'; // the value AT that address
    return 0;
}
```

```
5
0x7ffcc49e7ee0
5
```

> **!** Key idea
>
> `&` and `*` are opposites: `&` gets the address of an object, and `*` gets the object at an address. So `*(&x)` is just `x`.

1.4 One symbol, three meanings

The ampersand `&` means different things in different places:

Where	Meaning	Example
after a type	lvalue reference (Chapter 07)	`int& ref{x};`
before an expression	address-of	`std::cout << &x;`
between two expressions	bitwise AND (not covered here)	`int z{x & y};`

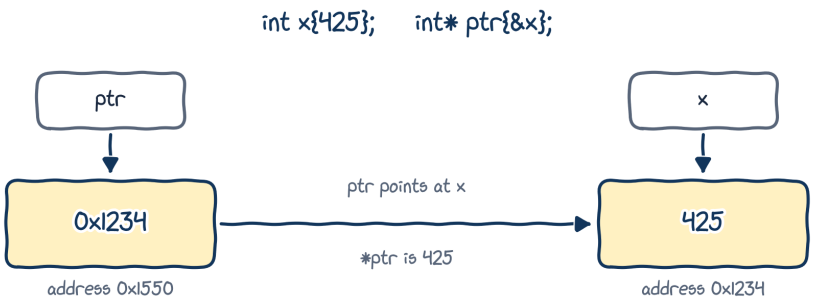
The asterisk `*` is similar: after a type it declares a pointer (next!), before an expression it dereferences, and between two expressions it multiplies.

2 Pointers

2.1 A pointer holds an address

A **pointer** is an **object** whose value is a memory address, usually the address of another object.

```cpp
double   // a double
double&  // a reference to a double
double*  // a pointer to a double: holds the address of a double
```



That's where pointers get their name: `ptr` holds the address of `x`, so `ptr` **points at** `x`.

2.2 Your first pointer

Listing 3 `pointer_example.cpp`

```cpp
#include <iostream>

int main() {
    int x{425};
    int* ptr{&x}; // ptr is a pointer to int, holding the address of x

    std::cout << "x lives at:   " << &x << '\n';
    std::cout << "ptr holds:    " << ptr << '\n';
    std::cout << "*ptr gives:   " << *ptr << '\n';
    std::cout << "ptr lives at: " << &ptr << '\n'; // a pointer is an object
    return 0;
}
```

```
x lives at:   0x7ffc45506a0c
ptr holds:    0x7ffc45506a0c
*ptr gives:   425
ptr lives at: 0x7ffc45506a10
```

`ptr` holds the address of `x`, `*ptr` is `x` itself, and `ptr` has an address of its own, because it's an object.

2.3 Declaring pointers

The `*` in a declaration belongs to **each** variable, not to the type:

```cpp
int* ptr1, ptr2;  // careful: ptr1 is a pointer, but ptr2 is a plain int!
int* ptr3, *ptr4; // both are pointers
```

Yet another reason to define one variable per line.

> * Best practice
>
> Write the asterisk next to the type, `int* ptr`, just as with references (`int& ref`).

2.4 Always initialize your pointers

Listing 4 `initialize_pointers.cpp`

```cpp
#include <iostream>

int main() {
    int x{5};

    int* ptr1;     // uninitialized: holds a garbage address (wild!)
    int* ptr2{};   // a null pointer: points at nothing
    int* ptr3{&x}; // points at x

    std::cout << *ptr3 << '\n';
    return 0;
}
```

- An uninitialized pointer is a **wild pointer**: dereferencing its garbage address is undefined behaviour.
- `int* ptr2{};` creates a **null pointer**: one that points at nothing. More on these soon.

> * Best practice
>
> Always initialize your pointers: with an address, or with `nullptr`.

2.5 Changing the pointer, or what it points at

Listing 5 `pointer_assignment.cpp`

```cpp
#include <iostream>

int main() {
    double x{42.0};
    double y{24.0};
    double* ptr{&x}; // ptr points at x

    *ptr = 10.0; // changes x: x becomes 10
    std::cout << "x = " << x << '\n';

    ptr = &y;    // change what ptr points at: now y
    *ptr = 12.0; // so this changes y
    std::cout << "y = " << y << '\n';
    return 0;
}
```

```
x = 10
y = 12
```

> ! Key idea
>
> - `ptr = &y;` assigns to the **pointer**: it points somewhere else.
> - `*ptr = 12.0;` assigns to the **object pointed at**.

2.6 Pointers behave much like references...

Listing 6 `pointers_behave_like_references.cpp`

```cpp
#include <iostream>

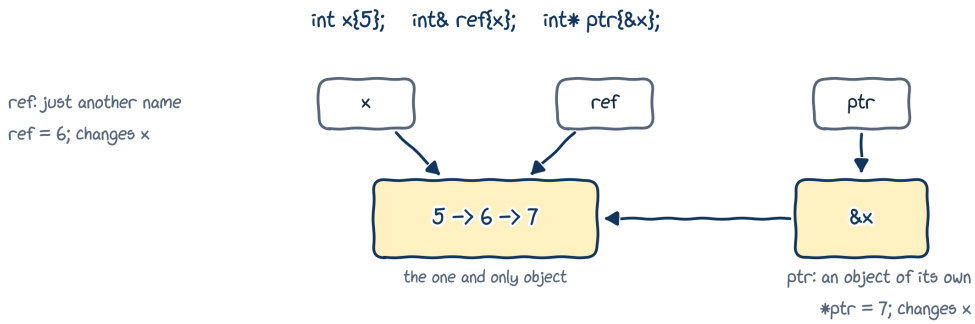
int main() {
    int x{5};
    int& ref{x};  // a reference to x
    int* ptr{&x}; // a pointer to x

    std::cout << x << ' ' << ref << ' ' << *ptr << '\n';

    ref = 6; // change x through the reference
    std::cout << x << ' ' << ref << ' ' << *ptr << '\n';

    *ptr = 7; // change x through the pointer
    std::cout << x << ' ' << ref << ' ' << *ptr << '\n';
    return 0;
}
```

```
5 5 5
6 6 6
7 7 7
```



2.7 ...but a pointer is not a reference

References	Pointers
must be initialized	don't have to be (but should be!)
are not objects	are objects
can't be reseated	can be made to point somewhere else
always refer to an object	can point at nothing (null)
"safe", apart from dangling	dangerous: wild, null and dangling pointers

2.8 The type of `&x`

`&x` doesn't produce a bare number: it produces a **pointer**. For an `int x`, `&x` has type `int*`:

Listing 7 `address_of_type.cpp`

```cpp
int main() {
    int x{5};
    int* ptr{&x};     // fine: &x is an int*
    double* dptr{&x}; // error: an int* is not a double*
    return 0;
}
```

```
error: cannot convert 'int*' to 'double*' in initialization
```

A pointer's type says what kind of object lives at the address, so an `int*` can't be used as a `double*`.

2.9 The size of a pointer

Listing 8 `size_of_pointers.cpp`

```cpp
#include <iostream>

int main() {
    char* char_ptr{};      // a char is 1 byte
    int* int_ptr{};        // an int is usually 4 bytes
    long double* ld_ptr{}; // a long double is often 16 bytes

    // But every pointer is the same size: it just holds an address
    std::cout << sizeof(char_ptr) << '\n';
    std::cout << sizeof(int_ptr) << '\n';
    std::cout << sizeof(ld_ptr) << '\n';
    return 0;
}
```

```
8
8
8
```

- A pointer is just an address, so all pointers are the same size, whatever they point at.
- That size depends on the machine: 8 bytes (64 bits) on a 64-bit system like mine, 4 bytes on a 32-bit one.

3 Null pointers

3.1 `nullptr`: pointing at nothing

Listing 9 `null_pointer.cpp`

```cpp
#include <iostream>

int main() {
    int* ptr{nullptr}; // a null pointer: points at nothing

    if (ptr == nullptr) { // check before dereferencing!
        std::cout << "ptr is null\n";
    }

    int x{5};
    ptr = &x;
    if (ptr) { // a non-null pointer counts as true
        std::cout << "ptr points at " << *ptr << '\n';
    }
    return 0;
}
```

```
ptr is null
ptr points at 5
```

- `nullptr` is a special value meaning "no address". A pointer holding it is a **null pointer**.
- A null pointer converts to `false` and any other pointer to `true`, so `if (ptr)` asks "does `ptr` point at something?"

3.2 Never dereference a null pointer

Listing 10 `null_dereference.cpp`

```cpp
#include <iostream>

int main() {
    int* ptr{nullptr};
    std::cout << *ptr << '\n'; // undefined behaviour!
    return 0;
}
```

```
$ ./null_dereference
Segmentation fault
```

There's no object at "no address", so `*ptr` is undefined behaviour. On most systems, the program crashes with a **segmentation fault**.

* Best practice

If a pointer might be null, check it before you dereference it.

3.3 Dangling pointers

Listing 11 `dangling_pointer.cpp`

```cpp
#include <iostream>

int main() {
    int* ptr{nullptr};
    {
        int x{5};
        ptr = &x;
    } // x is destroyed here: ptr now dangles

    std::cout << *ptr << '\n'; // undefined behaviour!
    return 0;
}
```

```
$ g++ -Wall dangling_pointer.cpp
warning: using dangling
pointer 'ptr' to 'x'
$ ./a.out
5
```

- Like a dangling reference, a **dangling pointer** still holds the address of an object that has been destroyed.
- It printed `5` here, but with `-O2` the same code printed `0`. Undefined behaviour: anything can happen. Thank you, `-Wall`!

4 Pass by address

4.1 Pass by address

A function can take a **pointer** parameter. The caller passes an **address**:

Listing 12 `pass_by_address.cpp`

```cpp
#include <iostream>

void print_value(const int* ptr) { // receives an address
    if (ptr == nullptr) {
        std::cout << "(no object)\n";
        return;
    }
    std::cout << *ptr << '\n';
}

int main() {
    int x{42};
    print_value(&x);      // pass the address of x
    print_value(nullptr); // "no object" is a valid argument
    return 0;
}
```

```
42
(no object)
```

`const int*` is a pointer to a `const int`: the function may read `*ptr` but not change it. With a plain `int*`, it could change the caller's `x` through `*ptr`.

4.2 References or pointers?

Pass by address can do what pass by reference does: avoid copies, and change the caller's object. The one extra thing it can do is mean **"no object at all"**, with `nullptr`.

> * Best practice
>
> Prefer pass by (const) reference. Use pass by address only when "no object" is a valid argument.

Pointers really earn their place when we create objects while the program runs (next chapter!) and when we work with class hierarchies. Remember the "proof" that scope is not lifetime in Chapter 02? That was pass by address.

5 Wrapping up

5.1 Summary

- Every object lives at an **address**. `&x` gives the address of `x`; `*ptr` gives the object at the address in `ptr`.
- A **pointer** (`int* ptr{&x};`) is an object that holds an address. Assign to `ptr` to point elsewhere; assign to `*ptr` to change the object pointed at.
- Always initialize pointers. `nullptr` means "points at nothing"; check before dereferencing.
- **Wild**, **null** and **dangling** pointers lead to undefined behaviour when dereferenced.
- References are safer; prefer them. Use a pointer when "no object" must be possible.

5.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Null pointers
2. Pointers and const: pointers to const, and const pointers
3. Pass by address
4. C-style arrays, and how they turn into pointers

Next up, Chapter 09: dynamic memory allocation, where pointers become essential.